Visual Website Optimizer

TEST PLAN DOCUMENT

Test Plan — VWO Login Dashboard

Prepared against the Product Requirements Document for app.vwo.com/login

Document	Test Plan
Product / Feature	VWO Login Dashboard
Application URL	app.vwo.com/login
Prepared By	QA Team
Version	1.0
Date	04 November 2026
Status	Draft — For Approval

TABLE OF CONTENTS

1. Objective
2. Scope
3. Inclusions
4. Test Environments
5. Defect Reporting Procedure
6. Test Strategy
7. Test Schedule
8. Test Deliverables
9. Entry and Exit Criteria
10. Tools
11. Risks and Mitigations
12. Approvals

1. Objective

The goal of this test plan is to ensure the quality, functionality, and reliability of the VWO Login Dashboard hosted at app.vwo.com/login.

The login dashboard is designed to handle secure user authentication for VWO's digital experience optimization platform, covering A/B testing, personalization, and analytics products for its user base.

- Track both leading and lagging indicators for the login experience's impact.
- Keep an eye on guardrail metrics to stop bad changes early on.

Sprint boot | My SQL | Documentation via Flagger

2. Scope

Scope of Test Plan for VWO Login Dashboard:

1. Functional Testing

- Verify the correctness and functionality of all login and authentication flows as per the requirements documentation.
- Test various scenarios for user login, session handling, and logout.
- Validate user authentication and authorization mechanisms for protected areas of the platform.

2. Data Validation Testing

- Ensure that the login form correctly validates input data, rejecting invalid requests.
- Test boundary values for input fields (email address, password) to check for any unexpected behavior.
- Validate the accuracy of the validation messages returned to the user.

3. Error Handling Testing

- Verify that appropriate error messages are returned for invalid login requests.
- Check error responses for sensitive information disclosure.
- Validate the login page's ability to handle unexpected errors gracefully.

4. Performance Testing

- Assess the login page's response time under normal and peak loads to identify potential bottlenecks.
- Measure the login throughput and scalability to handle concurrent requests.

5. Security Testing

- Conduct security assessments to identify vulnerabilities such as SQL injection, XSS, etc.
- Validate the login page's compliance with secure data transmission practices (e.g., HTTPS).
- Check for proper access controls to prevent unauthorized access to sensitive resources.

6. Integration Testing

- Verify interactions between the login dashboard and the downstream VWO services.
- Test data consistency across related modules.

7. Compatibility Testing

- Test the login dashboard on different platforms, browsers, and devices to ensure cross-compatibility.

8. Documentation Review

- Assess the clarity, completeness, and accuracy of the login dashboard documentation.
- Verify that the documentation is in sync with the actual behavior.

9. Load Testing

- Evaluate the login dashboard's behavior under high concurrent user loads to ensure stability.

10. Regression Testing

- Conduct regression testing after bug fixes or updates to ensure existing functionality remains intact.

11. Edge Case Testing

- Test extreme and boundary scenarios to identify potential issues.

12. Concurrency Testing

- Assess the login dashboard's behavior when multiple users attempt to log in simultaneously.

13. Ad Hoc Testing

- Perform exploratory testing to identify any hidden defects or usability issues.

14. Usability Testing

- Evaluate the login dashboard's user-friendliness and ease of use.

15. Continuous Integration and Deployment (CI/CD) Testing

- Validate the login dashboard's behavior within the CI/CD pipeline to ensure smooth deployments.

16. Performance Monitoring

- Implement monitoring to track login performance in real-time.

17. Backup and Recovery Testing

- Validate data backup and recovery procedures to ensure data integrity.

18. Internationalization Testing

- Test the login dashboard's behavior with different language settings.

19. Rate Limiting Testing

- Check the login dashboard's adherence to rate-limiting rules to prevent abuse.

20. Third-Party Integration Testing

- Validate any third-party integrations (enterprise SSO, social login) for smooth functioning.

> It's important to note that the scope of the test plan may evolve during the testing process based on feedback, changing requirements, or discoveries during testing. The scope should be reviewed and adjusted accordingly throughout the testing phase to ensure comprehensive coverage of the VWO Login Dashboard.

3. Inclusions

Create (POST) Operations:

- Test the login dashboard's ability to create new user sessions using valid input data.
- Verify that appropriate error responses are returned for invalid or missing data.
- Validate that newly created sessions are stored correctly in the system.

Read (GET) Operations:

- Test the login dashboard's ability to retrieve user and session information by various criteria.

- Verify that the dashboard returns the correct data in response to read requests.
- Test for correct handling of non-existent or invalid session IDs.

Update (PUT) Operations:

- Test the login dashboard's ability to update existing user account and session data with valid input.
- Verify that the dashboard rejects invalid update requests with appropriate error responses.
- Validate that the account data is correctly modified in the system after updates.

Delete (DELETE) Operations:

- Test the login dashboard's ability to terminate sessions (logout) by providing valid session IDs.
- Verify that the dashboard returns appropriate responses after a successful logout.
- Validate that the terminated sessions are removed from the system.

Boundary Testing:

- Test the login dashboard with minimum and maximum allowed values for input fields.
- Validate the behavior of the dashboard with values close to the boundaries.

Concurrency Testing:

- Test the login dashboard's behavior when multiple users try to log in simultaneously.
- Verify data consistency and handling of concurrent session modifications.

Data Validation:

- Test the login dashboard's response to various data validation scenarios (e.g., invalid characters, data types, mandatory fields).
- Verify that the dashboard handles validation errors appropriately.

Authentication and Authorization:

- Test login flows for both authenticated and unauthenticated users.
- Verify that only authorized users can access protected areas of the platform.

Error Handling:

- Test the login dashboard's response when invalid or malformed requests are made.
- Validate that appropriate error codes and messages are returned.

Security Testing:

- Test for security vulnerabilities during login operations (e.g., SQL injection, XSS).
- Verify that sensitive data is not exposed in responses.

Performance Testing:

- Evaluate the login dashboard's response time under normal and peak loads.
- Measure the throughput and scalability of the login dashboard.

Integration Testing:

- Verify the interaction and data consistency between login operations and other platform components.

Regression Testing:

- Perform regression tests after bug fixes or updates to ensure existing login functionalities remain intact.

Documentation Review:

- Assess the accuracy of the documentation related to login operations.

Load Testing:

- Evaluate the login dashboard's behavior and performance under high concurrent user loads.

Compatibility Testing:

- Test the login dashboard on different platforms, browsers, and devices.

Usability Testing:

- Evaluate the ease of using the login dashboard from a user's perspective.

Continuous Integration and Deployment (CI/CD) Testing:

- Validate the login operations within the CI/CD pipeline to ensure smooth deployments.

Rate Limiting Testing:

- Check the login dashboard's adherence to rate-limiting rules to prevent abuse.

Backup and Recovery Testing:

- Validate data backup and recovery procedures for login-related data.

4. Test Environments

- The operating systems and versions that will be used for testing, such as Windows 10, macOS, or Linux.
- The browsers and versions that will be tested, such as Google Chrome, Mozilla Firefox, or Microsoft Edge.
- The device types and screen sizes that will be used for testing, such as desktop computers, laptops, tablets, and smartphones.
- The network connectivity and bandwidth that will be available for testing, such as Wi-Fi, cellular, or wired connections.
- The hardware and software requirements for running the test cases, such as a specific processor, memory, or storage capacity.
- The security protocols and authentication methods that will be used to access the test environment, such as passwords, tokens, or certificates.
- The access permissions and roles of the team members who will be using the test environment, such as testers, developers, or stakeholders.

Name	Env URL
QA	app.vwo.com/login
Pre Prod	app.vwo.com/login

- Windows 10 – Chrome, Firefox and Edge
- Mac OS – Safari Browser
- Android Mobile OS – Chrome
- iPhone Mobile OS – Safari

5. Defect Reporting Procedure

- The criteria for identifying a defect, such as deviation from the requirements, user experience issues, or technical errors.
- The steps for reporting a defect, such as using a designated template, providing detailed reproduction steps, and attaching screenshots or logs.
- The process for triaging and prioritizing defects, such as assigning severity and priority levels, and assigning them to the appropriate team members for investigation and resolution.
- The tools and systems that will be used for tracking and managing defects, such as a defect tracking software or a project management tool.
- The roles and responsibilities of the team members involved in the defect reporting process, such as testers, developers, and the test lead.
- The communication channels and frequencies for updating stakeholders on the progress and status of defects.
- The metrics that will be used to measure the effectiveness of the defect reporting process, such as the number of defects found, the time taken to resolve them, and the percentage of defects that were successfully fixed.

Defect Process

POC	Owner
New Frontend	Devesh
Backend	Sonal
Dev Ops	Prajeeth

Tools – JIRA

6. Test Strategy

The first step is to create test scenarios and test cases for the various features in Scope.

While developing test cases, we'll use a number of test design techniques.

- Equivalence Class Partition
- Boundary Value Analysis
- Decision Table Testing
- State Transition Testing
- Use Case Testing

We also use our expertise in creating Test Cases by applying the below:

- Error Guessing
- Exploratory Testing
- We prioritize the Test Cases

Step 2: Our testing procedure when we receive a request for testing:

- First, we'll conduct smoke testing to see if the various and important functionalities of the application are working.
- We reject the build, if the Smoke Testing fails and will wait for the stable build before performing in depth testing of the application functionalities.
- Once we receive a stable build, which passes Smoke Testing, we perform in depth testing using the Test Cases created.

- Multiple Test Resources will be testing the same Application on Multiple Supported Environments simultaneously.

We then report the bugs in bug tracking tool and send dev. management the defect found on that day in a status end of the day email.

As part of the Testing, we will perform the below types of Testing:

- Smoke Testing and Sanity Testing
- Regression Testing and Retesting
- Usability Testing, Functionality & UI Testing
- We repeat Test Cycles until we get the quality product.

Step 3 – We will follow the below best practices to make our Testing better:

- **Context Driven Testing** – We will be performing Testing as per the context of the given application.
- **Shift Left Testing** – We will start testing from the beginning stages of the development itself, instead of waiting for the stable build.
- **Exploratory Testing** – Using our expertise we will perform Exploratory Testing, apart from the normal execution of the Test cases.
- **End to End Flow Testing** – We will test the end-to-end scenario which involves multiple functionalities to simulate the end user flows.

7. Test Schedule

Following is the test schedule planned for the project –

Task	Dates
Creating Test Plan	
Test Case Creation	
Test Case Execution	
Summary Reports Submission Date	

2 Sprints to Test the Application

8. Test Deliverables

- Test Plan
- Test Scenarios
- Test Cases
- Defect Reports
- Test Summary Reports

9. Entry and Exit Criteria

The below are the entry and exit criteria for every phase of the Software Testing Life Cycle:

Requirement Analysis

ENTRY CRITERIA:

- Once the testing team receives the Requirements Documents or details about the Project.

EXIT CRITERIA:

- List of Requirements are explored and understood by the Testing team.
- Doubts are cleared.

Test Execution

ENTRY CRITERIA:

- Test Scenarios and Test Cases Documents are signed-off by the Client.
- Application is ready for Testing.

EXIT CRITERIA:

- Test Case Reports, Defect Reports are ready.

Test Closure

ENTRY CRITERIA:

- Test Case Reports, Defect Reports are ready.

EXIT CRITERIA:

- Test Summary Reports.

10. Tools

The following are the list of Tools we will be using in this Project:

- JIRA Bug Tracking Tool
- Mind map Tool
- Snipping Screenshot Tool
- Word and Excel documents

11. Risks and Mitigations

The following are the list of risks possible and the ways to mitigate them:

- **Risk:** Non-Availability of a Resource
  Mitigation: Backup Resource Planning
- **Risk:** Build URL is not working
  Mitigation: Resources will work on other tasks
- **Risk:** Less time for Testing
  Mitigation: Ramp up the resources based on the Client needs dynamically

12. Approvals

Team will send different types of documents for Client Approval like below:

- Test Plan
- Test Scenarios
- Test Cases
- Reports

Testing will only continue to the next steps once these approvals are done.
